Algorithmique I - Corrigé du bilan 2023

Épreuve du 7 novembre 2023 · Questions 1 à 4 · LDA actualisé en 2026–2027

Les conditions, calculs et contrats de l'énoncé sont conservés. Les anciens mots-clés `algorithme`, `lire`, `écrire`, `retourner` et la flèche d'affectation sont transcrits en `void`/type de retour, `read`, `print`, `return` et `=`. Les données annoncées correctes sont supposées respecter le domaine de chaque question.

Question 1 - Pour commencer, traçons ! (4 points)

Source : bilan 2023, p. 1. `principal` lit `a=2`, `b=1`, `c=4`. Les numéros correspondent aux lignes de l'énoncé ; les trois modules disposent de variables locales distinctes.

Trace de `principal` (lignes 18 à 24).

Ligne	a	b	c
19 - Déclarations	-	-	-
20 - `read a, b, c`	2	1	4
21 - `a = b+c`	5	1	4
22 - `b = etape(a,c)`	5	17	4
23 - `a = double(b)`	34	17	4

Premier appel : `etape(5,4)` **(ligne 22 de `principal`).** Le test `x > 10` de la ligne 9 est faux : on exécute la ligne 12, pas la ligne 10.

Ligne de `etape`	x	y	z
7–8 - Paramètres et déclaration	5	4	-
12 - `z = x+y`	5	4	9
14 - `z = z+double(y)`	5	4	17

Retour (ligne 15) : `17`.

Appel imbriqué : `double(4)` **(ligne 14 de `etape`).**

Ligne de `double`	a	b
1–2 - Paramètre et déclaration	4	-
3 - `b = 2*a`	4	8

Retour (ligne 4) : `8`. La ligne 14 de `etape` calcule donc `9+8=17`.

Deuxième appel : `double(17)` **(ligne 23 de `principal`).** Il s'agit d'une nouvelle invocation : ses variables locales sont réinitialisées indépendamment du premier appel.

Ligne de `double`	a	b
1–2 - Paramètre et déclaration	17	-
3 - `b = 2*a`	17	34

Retour (ligne 4) : `34`.

État final de `principal` : `a=34`, `b=17`, `c=4`. **Affichage : aucun** ; ce code ne contient pas de `print`.

Question 2 - La fin de la pause (4 points)

Source : bilan 2023, p. 2. Ajouter 15 minutes à une heure de début `(H,M)`.

```
void finPause()
    int H, M, nouvelleHeure, nouvelleMinute
    read H, M
    nouvelleMinute = (M + 15) MOD 60
    nouvelleHeure = (H + (M + 15) DIV 60) MOD 24
    print nouvelleHeure, "h", nouvelleMinute
```

Exemples : `(10,15)` → `10h30` ; `(14,55)` → `15h10`. La retenue des minutes est `(M+15) DIV 60`. Le `MOD 24` assure aussi le passage de minuit : `(23,55)` → `0h10`.

Question 3 - Remplacer les chiffres (6 points)

Source : bilan 2023, p. 2. L'énoncé demande un algorithme qui **reçoit deux entiers en paramètres et retourne** le nouveau nombre. Le texte des exemples parle ensuite d'« afficher » : on retient le contrat explicite `return`, sans `print` à l'intérieur du module.

```
int remplacerChiffres(int nombre, chiffre)
    int c, d, u, resultat
    c = nombre DIV 100
    d = (nombre DIV 10) MOD 10
    u = nombre MOD 10
    if c == chiffre
        c = 0
    if d == chiffre
        d = 0
    if u == chiffre
        u = 0
    resultat = 100*c + 10*d + u
    return resultat
```

Préconditions : `100 ≤ nombre ≤ 999` et `1 ≤ chiffre ≤ 9`. **Contrôles :** `remplacerChiffres(231,3)=201` ; `(575,5)=70` ; `(981,3)=981`. La valeur 70 est un entier, pas la chaîne de trois caractères `"070"`.

Interprétation alternative : si l'on devait afficher le résultat, un algorithme principal pourrait faire `print remplacerChiffres(nombre, chiffre)`. Cette instruction n'est pas demandée dans la signature du module.

Question 4 - Allons nager (6 points)

Source : bilan 2023, p. 2 et affiche de la piscine. Ouverture de 9h à 12h (12h exclu), puis de 13h à 18h (18h exclu). Fermeture complète le dimanche et le mardi après-midi.

```
Boolean piscineOuverte(int jour, heure)
    Boolean ouvertMatin, ouvertApresMidi, resultat
    ouvertMatin = heure ≥ 9 AND heure < 12
    ouvertApresMidi = heure ≥ 13 AND heure < 18
    resultat = (jour ≠ 7) AND
               (ouvertMatin OR (jour ≠ 2 AND ouvertApresMidi))
    return resultat
```

Domaine : `jour` entre 1 (lundi) et 7 (dimanche), `heure` entre 0 et 23. **Essais de l'énoncé :** `(1,17)` → `true` ; `(2,17)` → `false` ; `(3,12)` → `false`. **Limites complémentaires :** mardi 11h → `true` ; dimanche 10h → `false` ; lundi 18h → `false`.